



Tema 2 - Fases del compilador

Cátedra: Organización del Computador II
Lic. Juan Pablo Moreno

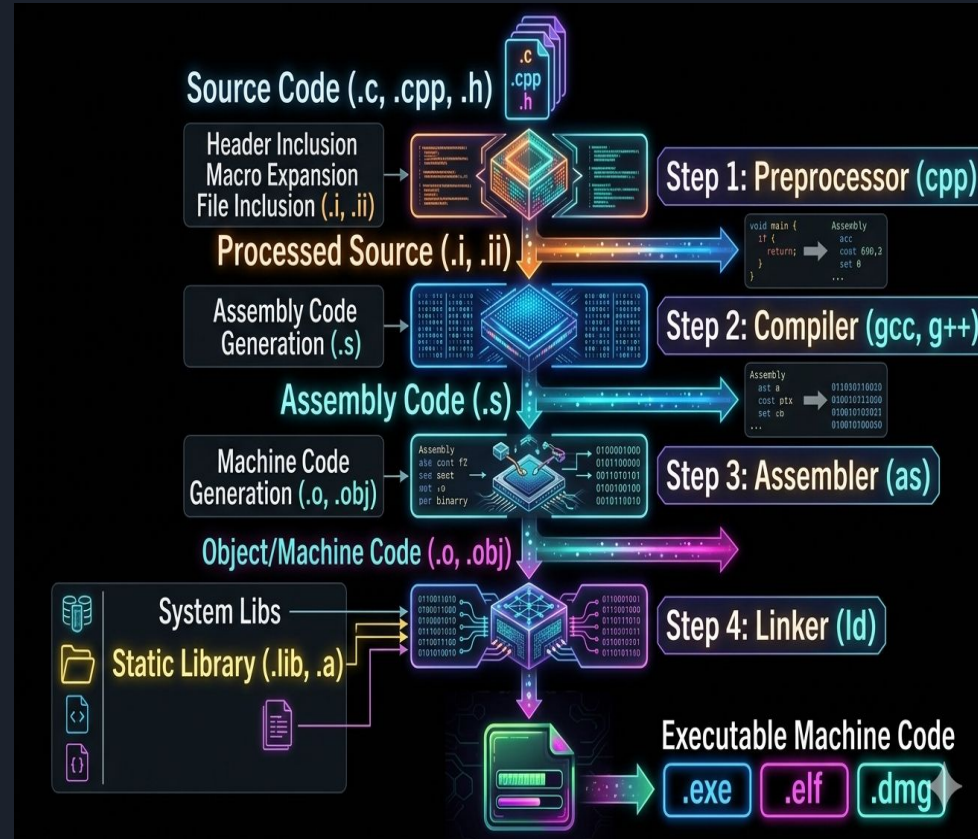


Contenido

1. Ensamblado
2. Vinculación
3. Carga y ejecución

Etapas de compilación

- Preprocesamiento: Prepara el código
 - #Include #define
 - Elimina comentarios
- Compilación
 - Convierte a .asm
- Ensamblaje
 - Lo Convierte en un archivo Objeto
- Vinculación
 - Se buscan las llamadas a funciones utilizadas en nuestro código
 - Se Reemplazan por las definiciones de cada función
- Carga y Ejecución



Ensamblado

- El objetivo del ensamblado es convertir de lenguaje ensamblador a código de máquina
- Los Ensambladores dividen su procedimiento en dos etapas o pasadas
- Esto se hace para evitar el problema de “forward reference”

DISASSEMBLY AND OBJECT CODE

29	0x401023:	ff 75 10	push dword [ebp + 0x10]
30	0x401026:	ff 75 0c	push dword [ebp + 0xc]
31	0x401029:	ff 75 08	push dword [ebp + 0x8]
32	0x40102c:	ff 15 48 72 40 00	call dword [0x407248 <DefWindow
33	0x401032:	e9 42 01 00 00	jmp 0x401179 <function_401000+0x
34	0x401037:	53	push ebx
35	0x401038:	56	push esi
36	0x401038:	8b 35 90 3e 42 00	mov esi, dword [0x423e90]
37	0x40103f:	8d 45 a4	lea eax, dword [ebp + 0xfffffa
38	0x401042:	57	push edi
39	0x401043:	50	push eax
40	0x401044:	ff 75 08	push dword [ebp + 0x8]

info bubble

ASSEMBLER PROCESS

miEtiqueta no se encuentra definida en la primera pasada

miEtiqueta

1 2

```

; código
buffsize equ 8192
jmp miEtiqueta
; código
miEtiqueta:
    xor eax, eax
    ; más código
    xor ebx, ebx
    
```

Primera pasada

Completa la tabla de símbolos

Se utiliza el ILC (Instruction length counter) para saber la posición de cada instrucción



Label	Opcode	Operands	Comments	Length	ILC
MARIA:	MOV	EAX, I	EAX = I	5	100
	MOV	EBX, J	EBX = J	6	105
ROBERTA:	MOV	ECX, K	ECX = K	6	111
	IMUL	EAX, EAX	EAX = I * I	2	117
	IMUL	EBX, EBX	EBX = J * J	3	119
	IMUL	ECX, ECX	ECX = K * K	3	122
MARILYN:	ADD	EAX, EBX	EAX = I * I + J * J	2	125
	ADD	EAX, ECX	EAX = I * I + J * J + K * K	2	127
STEPHANY:	JMP	DONE	branch to DONE	5	129

Note: Memory locations and specific opcodes may vary; this chart illustrates the assembly sequence and address calculation concept.

Primera pasada - Symbol Table

Sabiendo el ILC sé dónde se encuentra cada símbolo en memoria durante la ejecución

Esto me va a permitir durante la segunda pasada reemplazar las apariciones por la posición en memoria que tendrá en ejecución

Otras tablas que se usan durante el ensamblado son: opcode Table y pseudoinstruction Table



Tabla de símbolos

Symbol	Value
MARIA	100
ROBERTA	111
MARILYN	125
STEPHANY	129

Compilador / Ensamblador - Tabla de Símbolos de Usuario

Primer pasada - Opcode Table

- Ayuda a reemplazar cada instrucción por un valor hexadecimal
- Además posee un instruction class que indica a qué procedimiento llamar para ensamblar familias de instrucciones
- Algoritmo primera pasada: <https://pastebin.com/iWJTd3Fx>



X86 Opcode Reference (Sample)

Opcode	First operand	Second operand	Hexadecimal opcode	Instruction length	Instruction class
AAA	-	-	37	1	6
ADD	EAX	immed32	05	5	4
ADD	reg	reg	01	2	19
AND	EAX	immed32	25	5	4
AND	reg	reg	21	2	19

Segunda pasada

- Voy leyendo el código otra vez línea a línea
- Convertir las instrucciones a código máquina
- Si encuentro una etiqueta ya conozco su valor en memoria
- Algoritmo Segunda Pasada:
<https://pastebin.com/ijP96Xjr>

Errores que detecta:

- Un símbolo usado no está definido
- Un símbolo está definido más de una vez
- El nombre de una operación no está definida en la “Opcode Table”
- Faltan o hay operandos de más para el opcode en cuestión
- Uso ilegal de registros (jmp eax)
- Falta sentencia END

La tabla de Símbolos

- Es esencialmente una tabla clave, valor
- La forma más sencilla de representarla es un arreglo de n tuplas
- Sin embargo esa implementación sería $O(n)$.
- ¿Hay alguna opción mejor?

TABLA DE DATOS DE USUARIOS (CONCEPTUAL)
Resolución de Símbolos de Datos de Usuario

Nombre	Valor
Andy	14.025
Anton	31.253
Cathy	65254
Dick	54.185
Erik	47357
Frank	14332
Gerrit	44534
Hans	75544
Henri	17097
Jaco	64533
Maarten	23267
Reind	63453
Roel	76764
Willem	34544
Wiebren	34344

La tabla de Símbolos

- Es esencialmente una tabla clave, valor
- La forma más sencilla de representarla es un arreglo de n tuplas
- Sin embargo esa implementación sería $O(n)$.
- ¿Hay alguna opción mejor?
- Convierto cada clave en ASCII en índice



La tabla de Hash

k es el tamaño de nuestra Hash table

Si k es cercano a n => la complejidad de búsqueda es $O(1)$ en el caso promedio

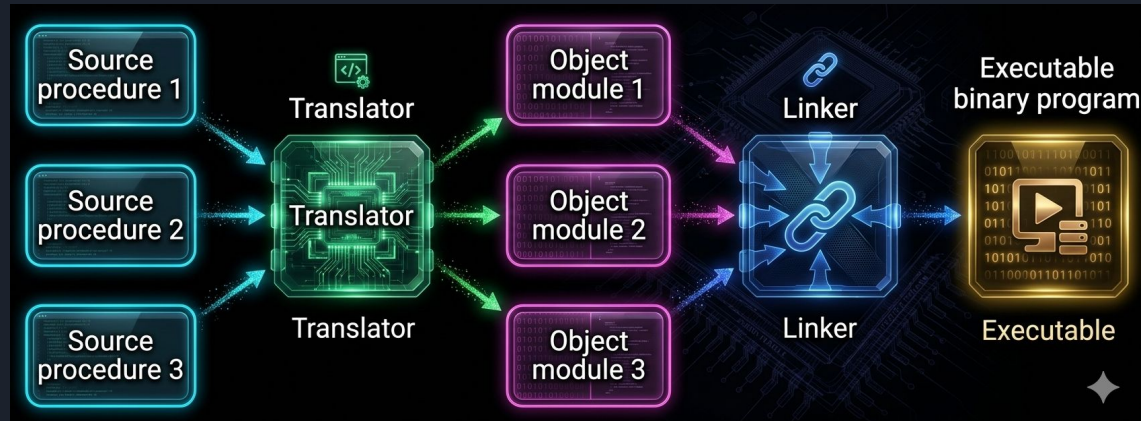


HASH TABLE CON RESOLUCIÓN DE COLISIONES (CONCEPTUAL)



Vinculación

- Separar el ensamblado por módulos
 - Si cambio algo en uno no tengo que recompilar el resto
- Todos los módulos arrancan en la dirección virtual 0
 - Trabajan en espacios de memoria separados
- Es la tarea del Linker unir estos espacios



Vinculación - Pasos

Construye una tabla de todos los módulos y sus longitudes

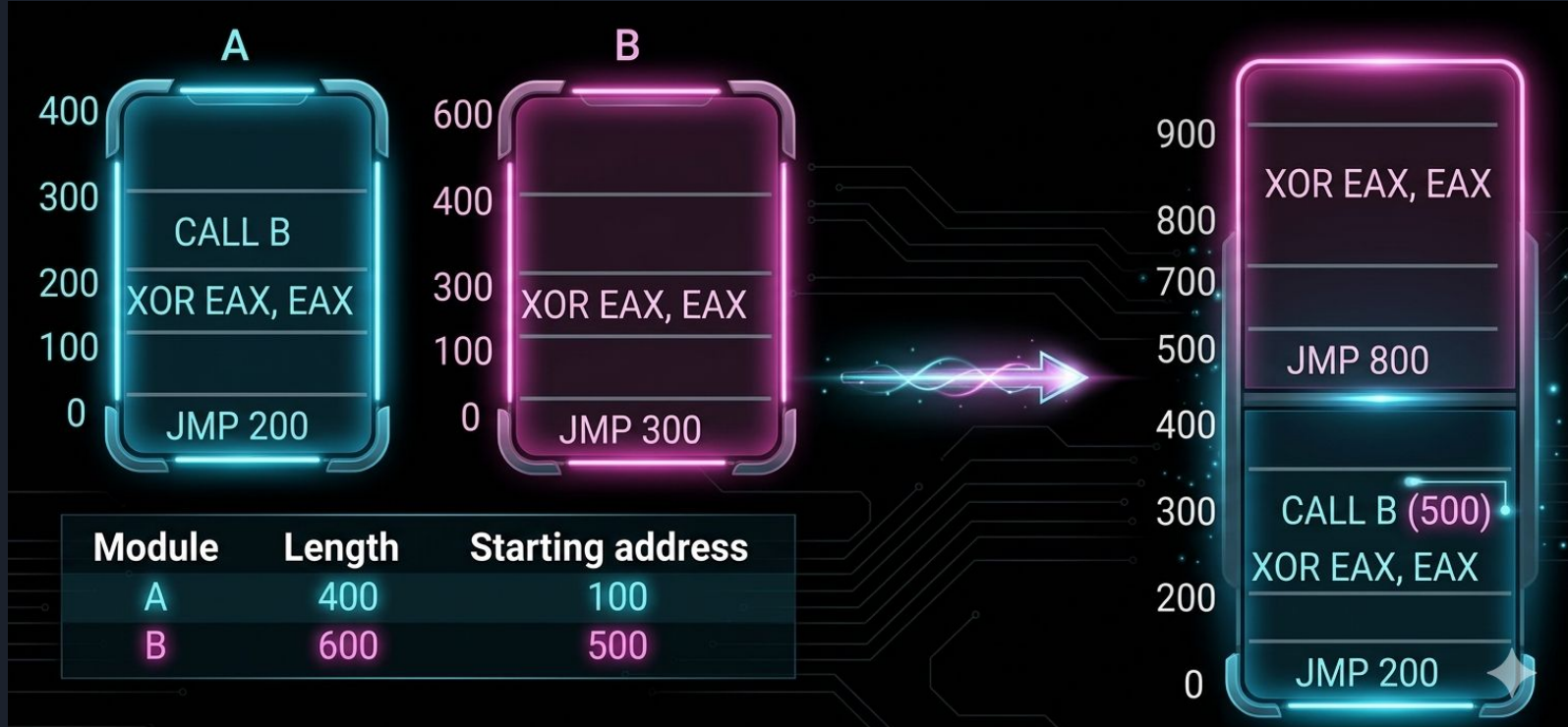
Basado en esta tabla le asigna una dirección de inicio a c/u

Busca todas las instrucciones que hacen referencia a memoria y les suma una constante offset = dirección de inicio del módulo.

Busca todas las instrucciones que hagan referencia a otros módulos e inserta su dirección final correcta

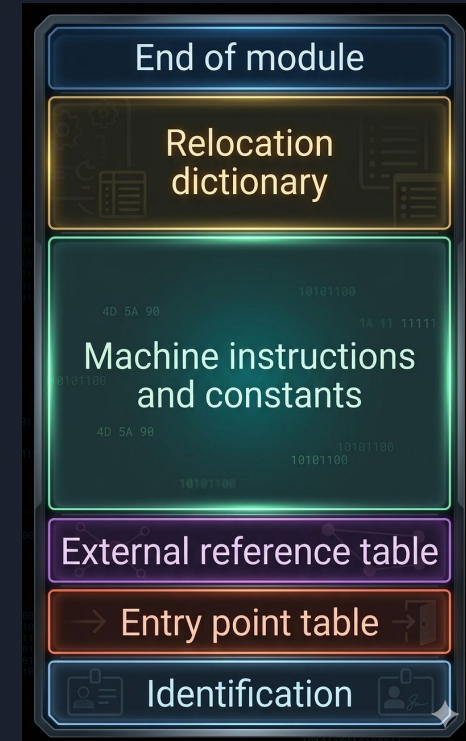


Vinculación - Pasos



Vinculación - Estructura de los módulos

- Identification: Nombre del módulo etc.
- Entry Point Table: Contiene las funciones que pueden ser llamadas por otros módulos (PUBLIC) y su dirección
- External Reference Table: Símbolos usados por este módulo que pertenecen a otros módulos.
- Única parte cargada durante la ejecución
- Contiene las direcciones de memoria que deberán ser relocalizadas por el linker
- Checksum





Carga

Carga las instrucciones del código ensamblado y con link en memoria

¿Pero qué pasa si se me acaba la ram y tengo que sacar una parte de mi programa?

Si se usa paginación como la page table me abstraeré de esto las direcciones virtuales dentro de mi programa no cambian



Vinculación Dinámica

¿Que pasa si hay un programa que tiene una librería que se usa poco? ¿Tengo que cargar todo en memoria?

La vinculación dinámica permite cargar los módulos a demanda.

También sirve para compartir módulos muy frecuentes entre procesos (librerías)

Tiene distintas implementaciones según SO

- Windows: DLL (Dynamic Link Library)
- UNIX: .SO

Vinculación Dinámica

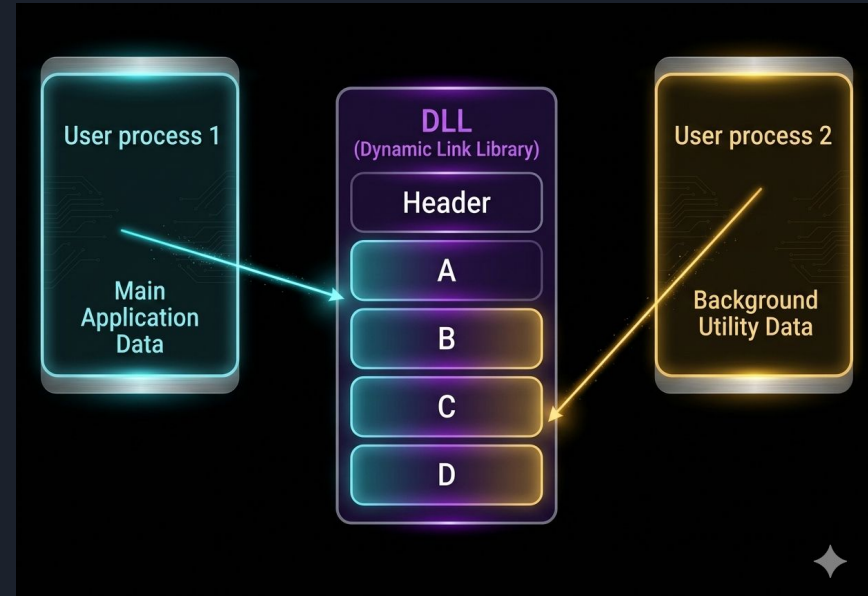
El módulo tiene listado que otros módulos necesita

Durante la carga el SO se fija qué tipo de linkeo es necesario

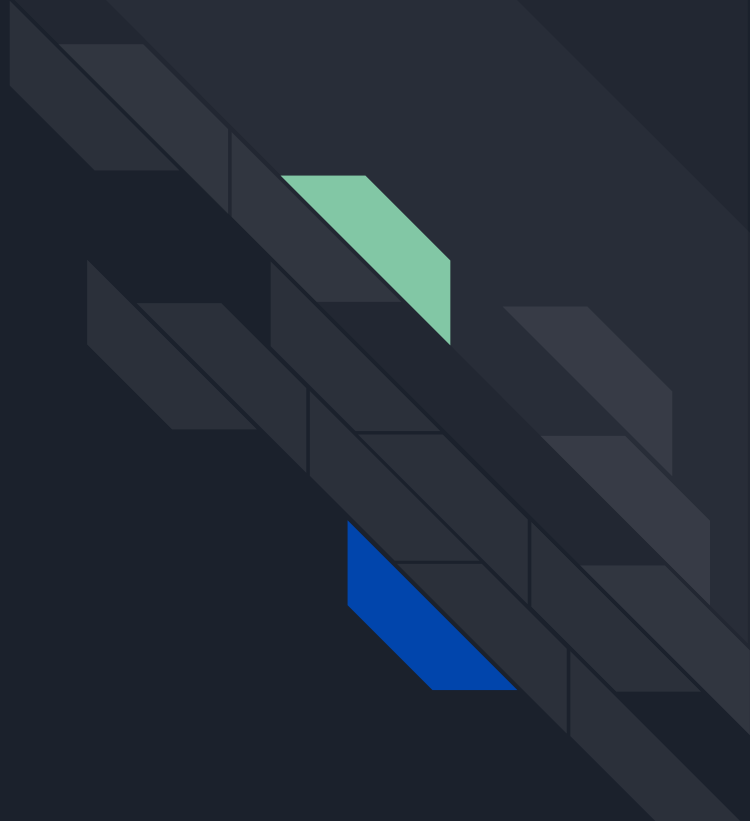
- Precarga lo carga al comienzo
- A demanda lo carga solo cuando es necesario

Permite que varios programas compartan una librería

Tiene la contra de que genera dependencias



Gracias!

An abstract graphic on the right side of the slide, consisting of several dark gray rectangular blocks arranged in a staggered, 3D-like pattern. Two blocks are highlighted: a light green one in the upper middle and a blue one in the lower right.